

Activity Logs Feature

Full Technical Documentation

Project: Fideon OS Platform

Feature: Audit Trail & Activity Log System

Roles Covered: global_admin · admin · user · viewer · guest

Date: 2026-03-16

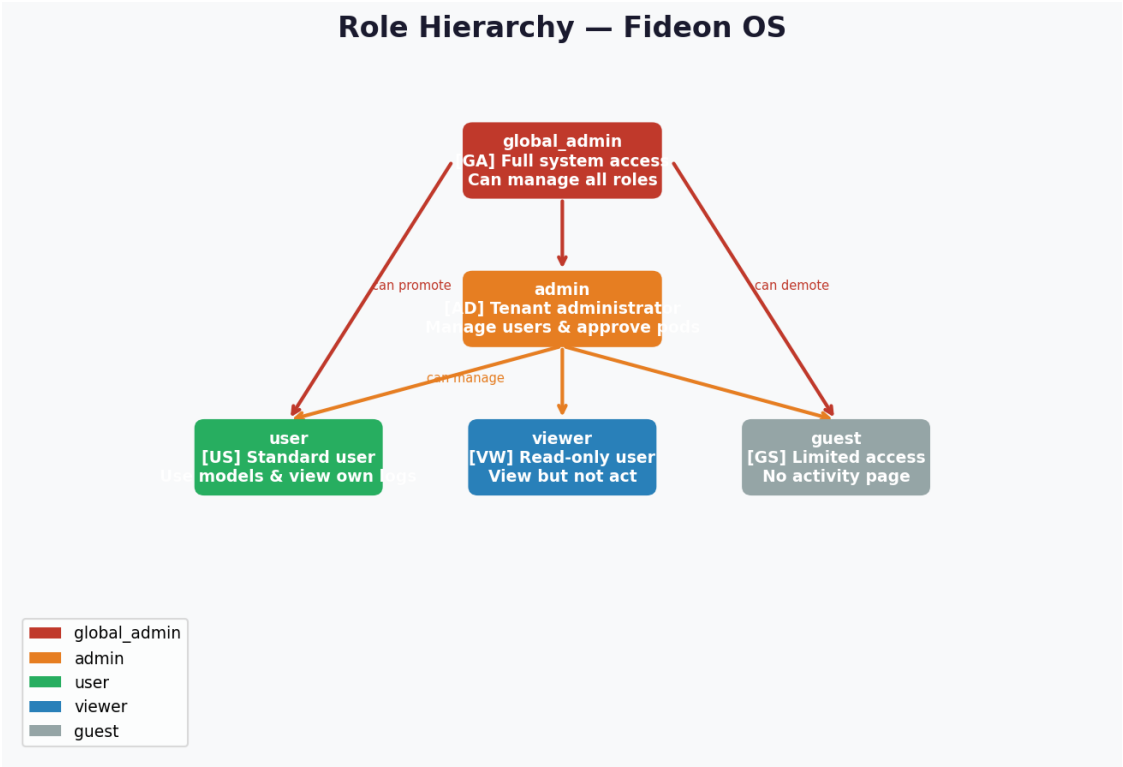


Figure 1 — Role Hierarchy Overview

1. Feature Overview

The **Activity Logs** system provides a tamper-evident, role-scoped audit trail of all significant actions taken across the Fideon OS platform. It is built on **two Supabase PostgreSQL tables** (*auth_audit* and *audit_logs*), enforced by PostgreSQL **Row-Level Security (RLS)** policies, and surfaced to users via the **activity** frontend page.

Core Design Principles

- **Tamper-Evidence:** Each *auth_audit* row stores a SHA-256 *integrity_hash* computed from non-PII fields. Any post-write modification invalidates the hash.
- **Privacy (PII Protection):** The email field is stored in the row but deliberately excluded from the integrity hash. Backend server logs use PII scrubbing for fields like password, token, email, phone, ssn.
- **Least Privilege:** Each role sees only what they are authorized to see via RLS policies. Users see only their own rows; admins see their org; *global_admin* sees all.
- **ATNA Compliance:** Action codes (C/R/U/D/E) and outcome codes (0/4/8/12) follow the ATNA (Audit Trail and Node Authentication) healthcare audit standard.
- **Immutability:** Audit rows are protected by PostgreSQL BEFORE UPDATE / BEFORE DELETE triggers (*prevent_audit_modification()*) that raise an exception for any modification attempt — including *service_role* and direct DB connections. This enforces EU AI Act Art.12, SOC2 CC7.2, and NAIC compliance.

2. Role Definitions & Hierarchy

The platform uses a **five-tier role model** stored as a PostgreSQL enum (app_role) and managed through the user_roles table. Only **global_admin** can assign or change roles.

Role	Color Code	Permissions Level	Activity Page
global_admin	Red (#c0392b)	Full access — sees ALL logs from ALL roles	YES All rows
admin	Orange (#e67e22)	Sees admin/user/viewer/guest — NOT global_admin	YES Filtered rows
user	Green (#27ae60)	Standard user — own logs only	YES Own rows only
viewer	Blue (#2980b9)	Read-only — own logs only	YES Own rows only
guest	Grey (#95a5a6)	Most restricted — no activity page access	NO Blocked

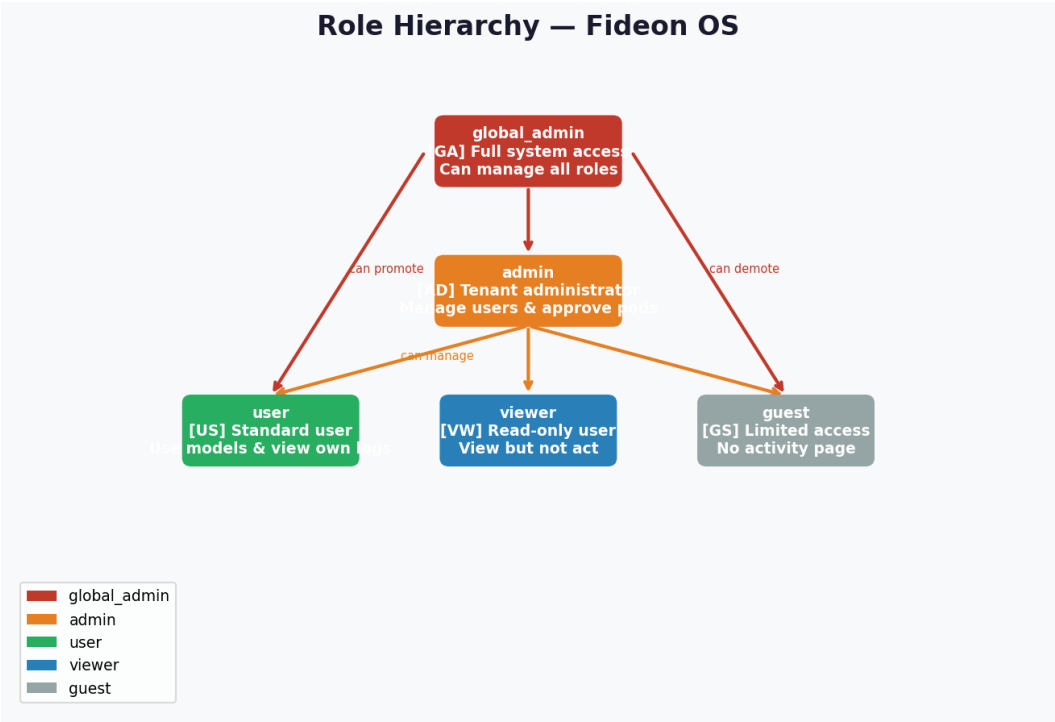


Figure 2 — Role Hierarchy with Promotion Paths

3. Entity-Relationship (ER) Diagram

The diagram below shows all seven database tables and their relationships. The **auth_audit** and **audit_logs** tables both reference **auth.users** via **user_id**. Device-related logs are linked to the **devices** table.

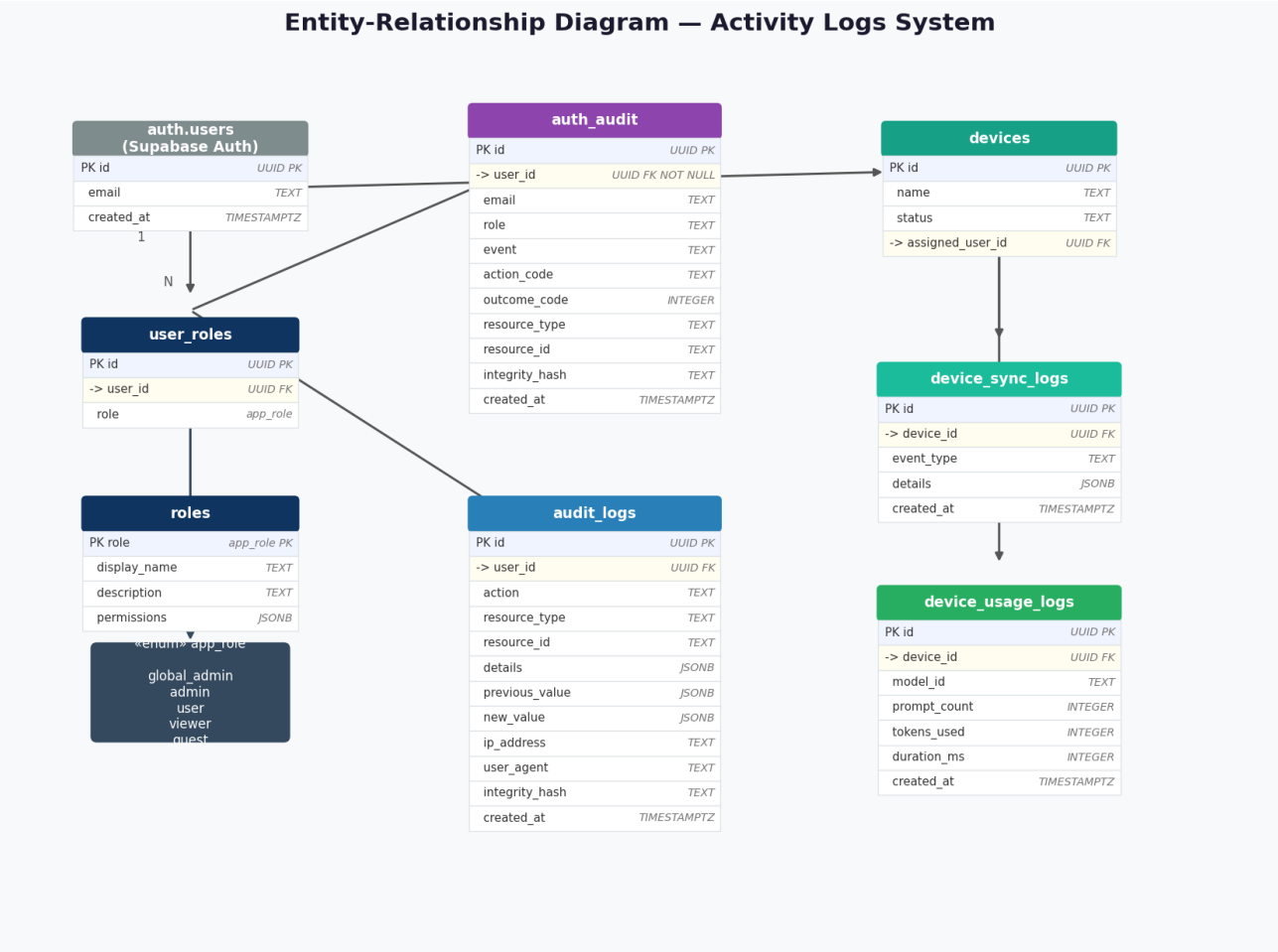


Figure 3 — Full Entity-Relationship Diagram

4. Database Schema — Column-Level Detail

Detailed column specifications for the two primary audit tables and the role assignment table, including data types, constraints, and descriptions.

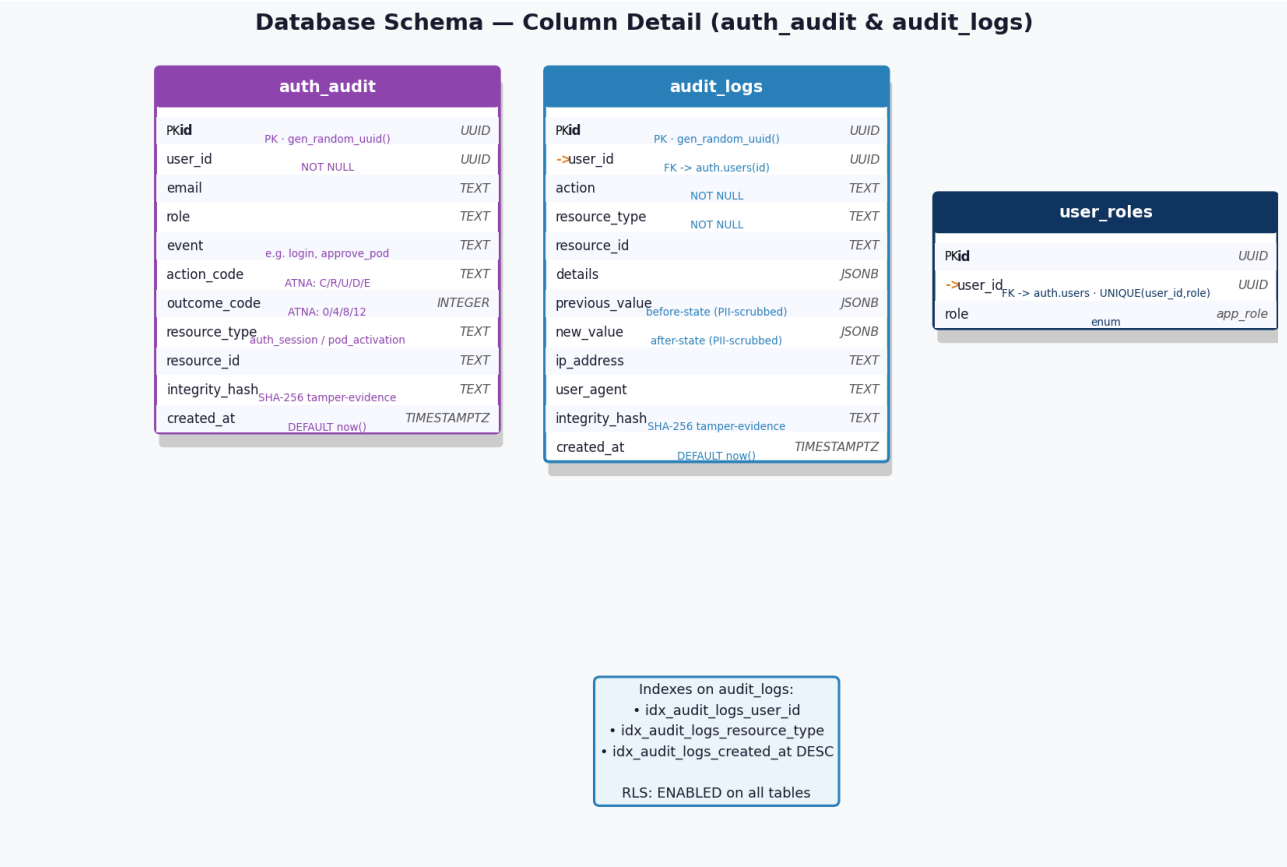


Figure 4 — Schema Column Detail Diagram

4.1 auth_audit — SQL Definition

```
CREATE TABLE public.auth_audit (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL REFERENCES auth.users(id),  
  email       TEXT,  
  role        TEXT,  
  event       TEXT,  
  action_code TEXT,         -- ATNA: C/R/U/D/E  
  outcome_code INTEGER,    -- ATNA: 0/4/8/12  
  resource_type TEXT,  
  resource_id TEXT,  
  integrity_hash TEXT,      -- SHA-256 tamper-evidence  
  created_at  TIMESTAMPTZ DEFAULT now()  
);
```

4.2 audit_logs — SQL Definition

```
CREATE TABLE public.audit_logs (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          REFERENCES auth.users(id),  
  action       TEXT          NOT NULL,
```

```
resource_type TEXT NOT NULL,
resource_id TEXT,
details JSONB,
previous_value JSONB, -- before-state, PII-scrubbed (added migration 20260316000002)
new_value JSONB, -- after-state, PII-scrubbed (added migration 20260316000002)
ip_address TEXT,
user_agent TEXT,
integrity_hash TEXT, -- SHA-256 tamper-evidence (added migration 20260316000001)
created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_audit_logs_user_id ON public.audit_logs (user_id);
CREATE INDEX idx_audit_logs_resource_type ON public.audit_logs (resource_type);
CREATE INDEX idx_audit_logs_resource_id ON public.audit_logs (resource_id) WHERE resource_id IS NOT NULL;
CREATE INDEX idx_audit_logs_created_at ON public.audit_logs (created_at DESC);
```

5. Row-Level Security (RLS) Policies

All audit tables have RLS **enabled**. The policies enforce that each role can only access the rows it is authorized to see. There are **no UPDATE or DELETE policies** and additionally, **database-level BEFORE UPDATE / BEFORE DELETE triggers** (prevent_audit_modification()) make audit records truly immutable — even service_role and direct Postgres connections cannot modify rows. The has_role(uid, 'admin') helper function is **hierarchical** and returns true for both admin and global_admin, so global_admin retains full access to device logs and audit_logs via those policies.

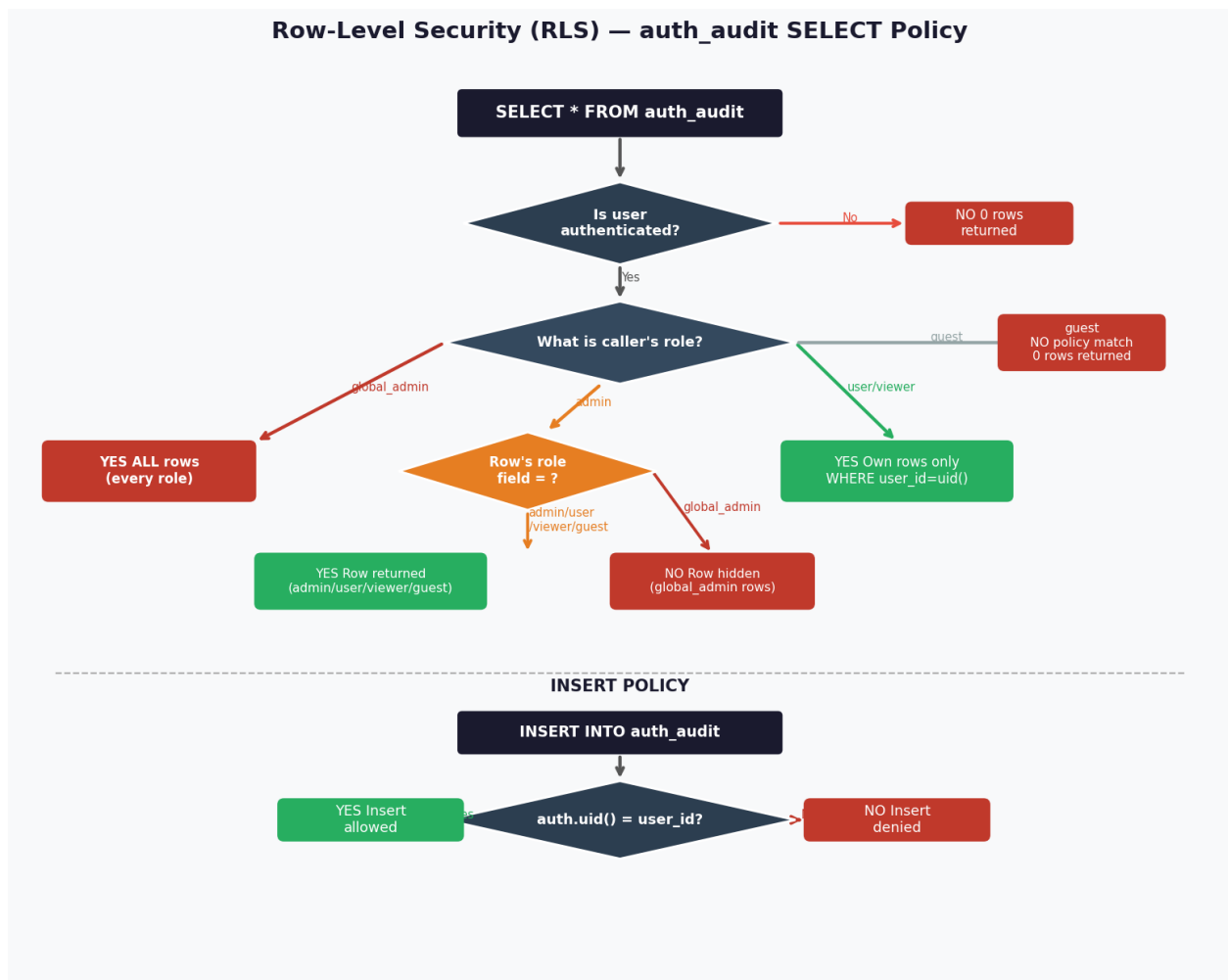


Figure 5 — RLS Policy Decision Flow

Table	Policy Name	Operation	Who	Condition
auth_audit	Users insert own	INSERT	Any auth user	auth.uid() = user_id
auth_audit	Users see own	SELECT	user/viewer	user_id = auth.uid()
auth_audit	Admins see user+admin	SELECT	admin	role IN (admin,user,viewer,guest)
auth_audit	Global admins see all	SELECT	global_admin	No restriction
audit_logs	System can insert	INSERT	Any	true
audit_logs	Users see own	SELECT	Any	auth.uid() = user_id
audit_logs	Admins view all	SELECT	admin/global	has_role(uid, 'admin') [hierarchical: in

device_sync_logs	Admins only view	SELECT	admin/global	has_role(uid, 'admin') [hierarchical]
device_usage_logs	Admins only view	SELECT	admin/global	has_role(uid, 'admin') [hierarchical]

6. Data Flow — How Logs Are Created

Every audit row follows a **five-step pipeline**: the user triggers an action in the frontend, the relevant component captures the event, auditHash.ts computes a SHA-256 integrity hash, the Supabase RLS INSERT policy validates the caller, and finally the row is persisted in the auth_audit table.

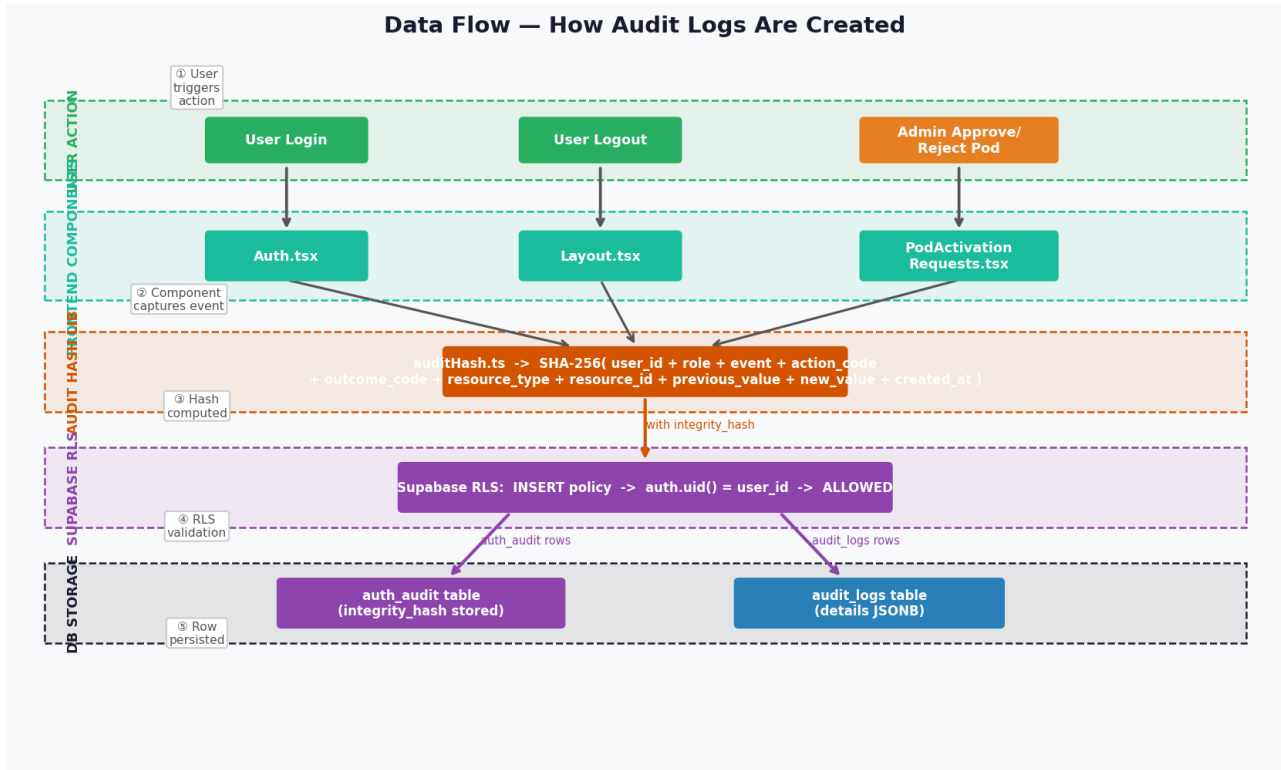


Figure 6 — Audit Data Flow with Swim Lanes

7. Sequence Diagrams

7.1 Login Audit Event

When a user successfully signs in, Auth.tsx immediately captures the timestamp, calls auditHash.ts to compute the integrity hash, then inserts a login event row into auth_audit.

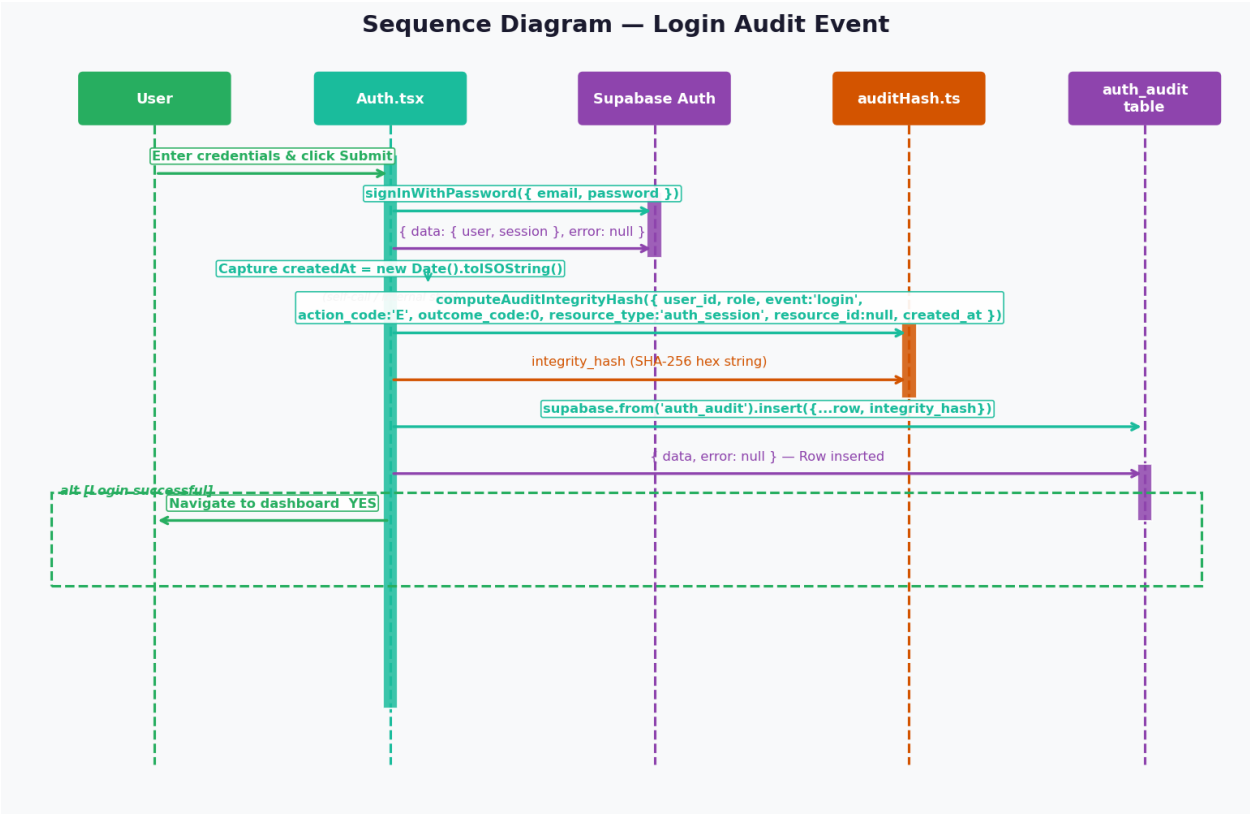


Figure 7 — Login Audit Sequence

7.2 Admin Approves Pod Activation

When an admin clicks Approve on a pod request, the component first updates the pod_activation_requests table, then computes the audit hash and writes an approve_pod:{model_id} event to auth_audit.

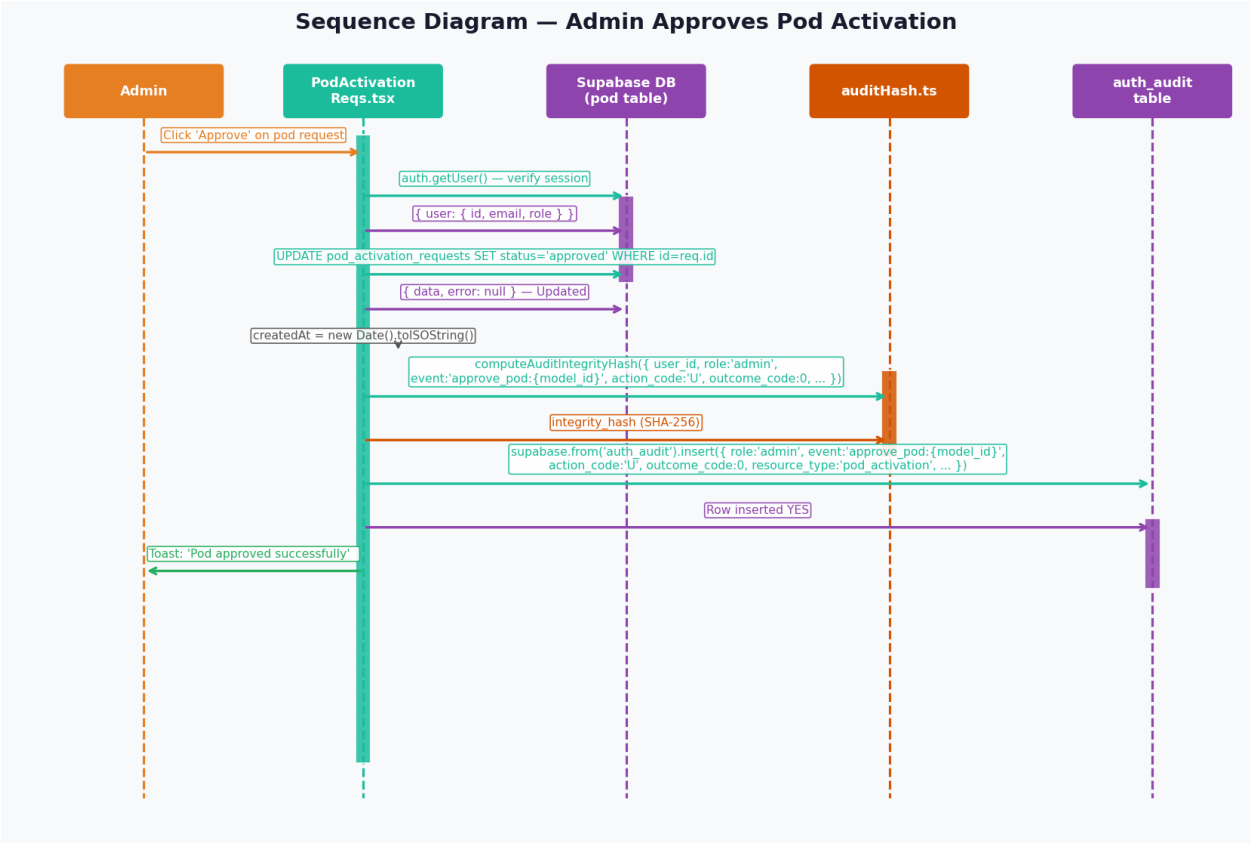


Figure 8 — Admin Approve Pod Sequence

7.3 Viewing Activity Logs

When any authenticated user navigates to **/activity**, Activity.tsx issues a SELECT query. The Supabase RLS layer automatically filters rows based on the caller's role — global_admin sees all, admin sees most, user/viewer sees only their own rows, and guest receives nothing.

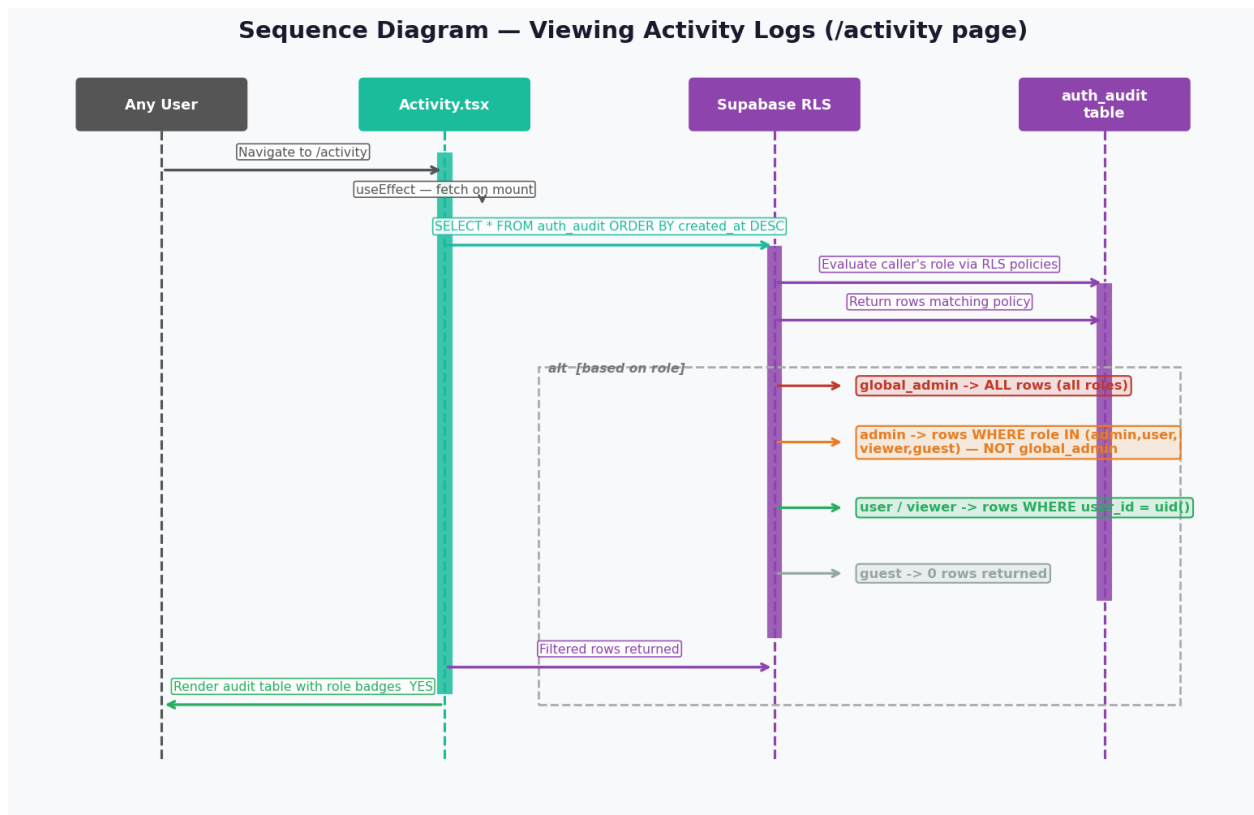


Figure 9 — View Activity Logs Sequence

8. Audit Integrity Hash — Tamper Evidence

Every row in `auth_audit` includes an `integrity_hash` field computed using **SHA-256** via the Web Crypto API (`crypto.subtle.digest`). The hash is computed from **eight non-PII fields**. The email field is deliberately **excluded** from the hash to protect personally identifiable information.

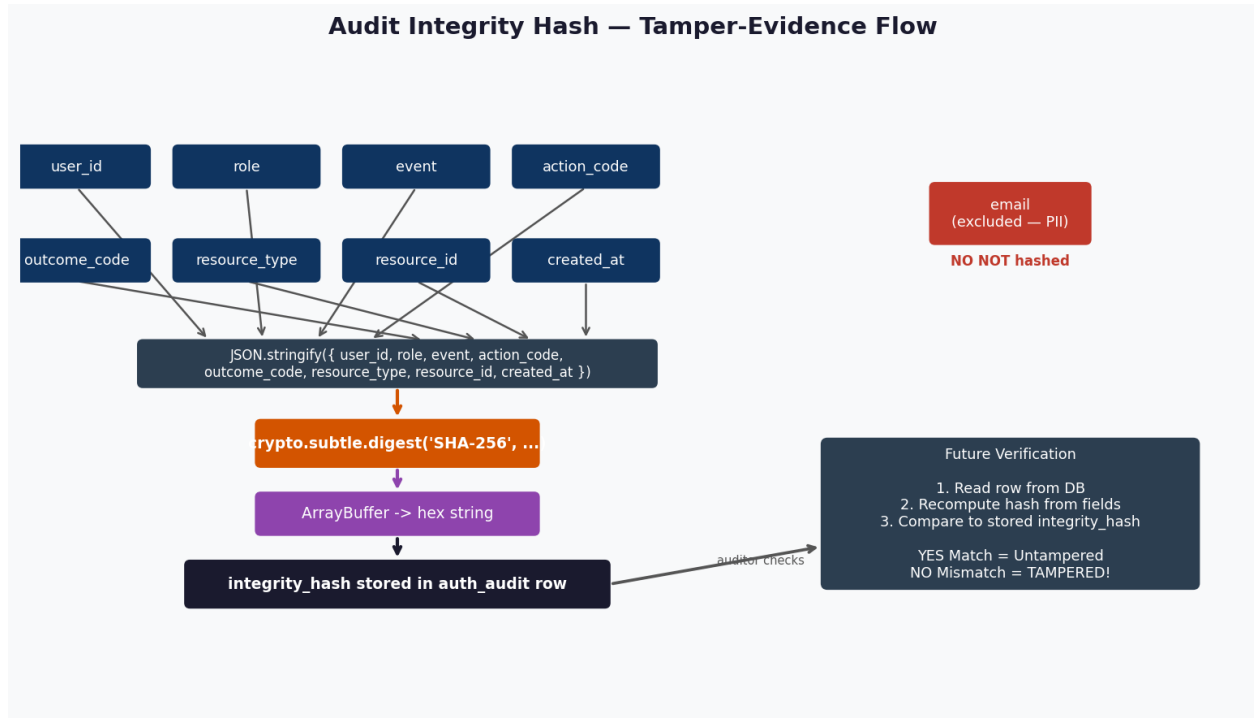


Figure 10 — Integrity Hash Computation & Verification

Hash Input Fields

- **user_id** — the UUID of the acting user
- **role** — the user's role at time of action
- **event** — the event name (login, logout, approve_pod, etc.)
- **action_code** — ATNA action code (C/R/U/D/E)
- **outcome_code** — ATNA outcome code (0/4/8/12)
- **resource_type** — resource category (auth_session, pod_activation)
- **resource_id** — specific resource identifier (**nullable**: passed as null for auth_session events such as login / login_failed / logout)
- **created_at** — ISO 8601 timestamp

Excluded from hash: email (PII — stored in row but not hashed)

9. Frontend Component Architecture

The frontend is a React/TypeScript single-page application. Role detection is centralized in the useUserRole hook. All audit writes flow through auditHash.ts. Route protection is enforced by ProtectedRoute.tsx.

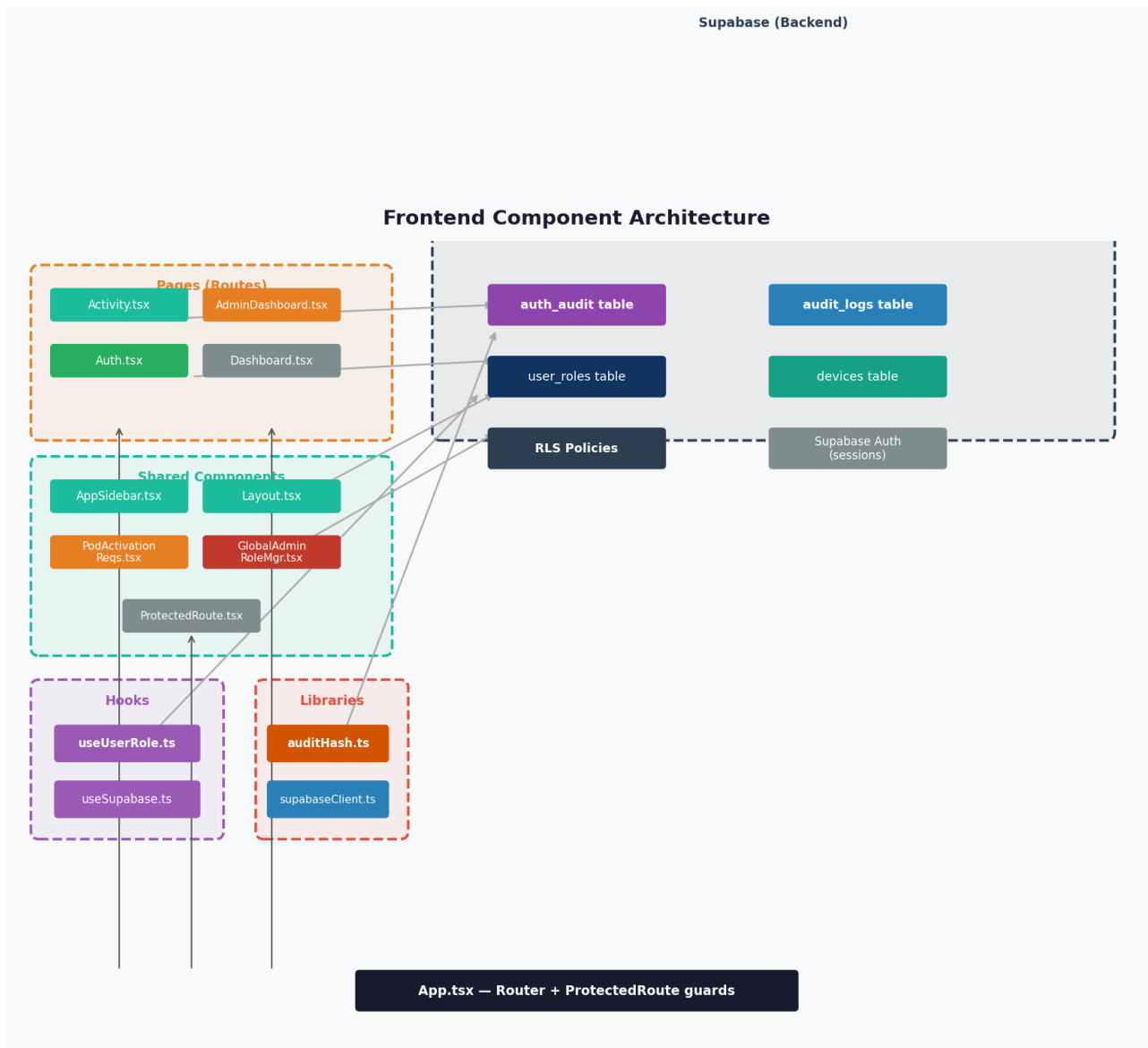


Figure 11 — Frontend Component Architecture Diagram

10. Access Control Matrix

The matrix below provides a complete overview of which roles can perform which operations across the activity logs system and related features.

Access Control Matrix — Activity Logs Feature					
Permission / Capability	global_admin	admin	user	viewer	guest
Access /activity page	YES	YES	YES	YES	NO
View own audit rows	YES	YES	YES	YES	NO
View all 'user' role rows	YES	YES	NO	NO	NO
View all 'admin' role rows	YES	YES	NO	NO	NO
View all 'viewer' role rows	YES	YES	NO	NO	NO
View all 'guest' role rows	YES	YES	NO	NO	NO
View 'global_admin' rows	YES	NO	NO	NO	NO
View ALL audit_logs	YES	YES	NO	NO	NO
View own audit_logs	YES	YES	YES	YES	NO
Approve / Reject Pods	YES	YES	NO	NO	NO
List all users (API)	YES	YES	NO	NO	NO
Create users (API)	YES	YES	NO	NO	NO
Set user roles (API)	YES	NO	NO	NO	NO
View device_sync_logs	YES	YES	NO	NO	NO
View device_usage_logs	YES	YES	NO	NO	NO
Access Admin Dashboard	YES	YES	NO	NO	NO

Figure 12 — Full Access Control Matrix

11. Event Types & ATNA Codes

The table below lists all audit event types currently implemented, along with the ATNA-standard action and outcome codes, and the resource types they apply to.

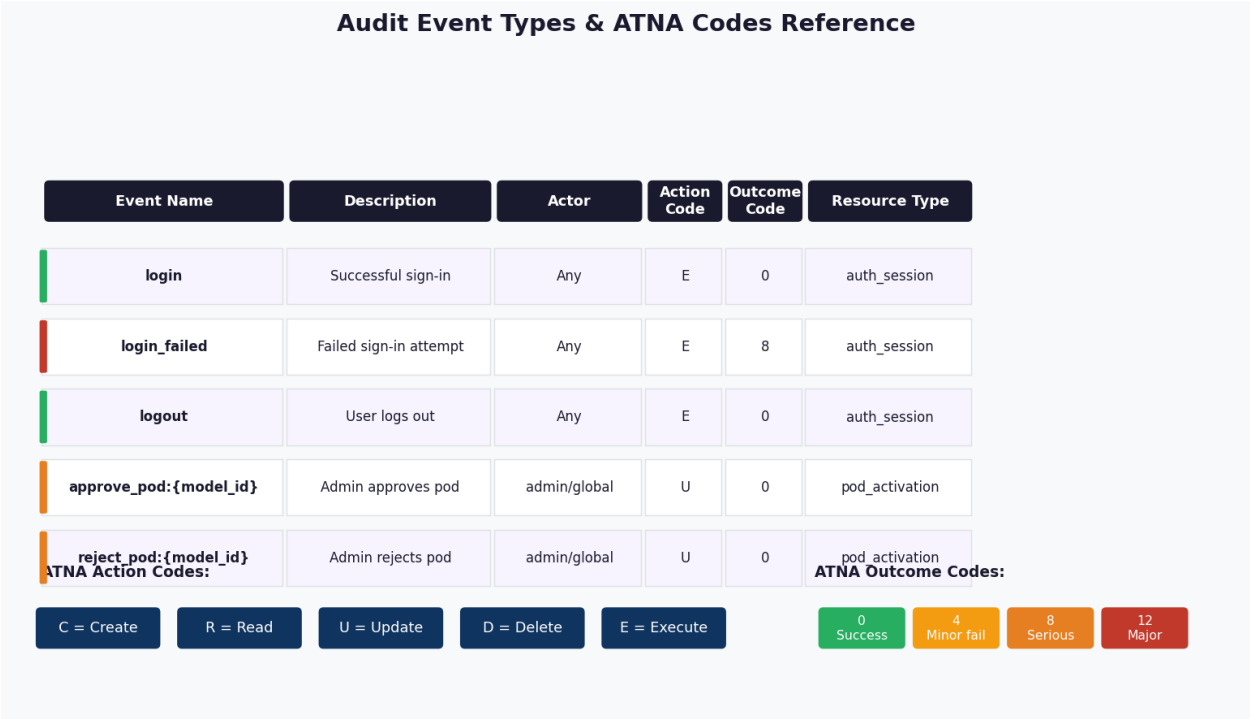


Figure 13 — Event Types & ATNA Codes Reference

Event Types Table

Event	Description	Actor	action_code	outcome_code	resource_type
login	Successful sign-in	Any	E	0	auth_session
login_failed	Failed sign-in attempt	Any	E	8	auth_session
logout	Sign-out click	Any	E	0	auth_session
approve_pod:{model_id}	Admin approves pod	admin/global	U	0	pod_activation
reject_pod:{model_id}	Admin rejects pod	admin/global	U	0	pod_activation

ATNA Code Reference

Code	Meaning	Used For	Type
C	Create	Creating new resources	Action
R	Read	Reading/viewing records	Action
U	Update	Modifying existing records	Action
D	Delete	Deleting records	Action

E	Execute	Executing operations (login/logout)	Action
0	Success	Operation completed normally	Outcome
4	Minor failure	Low severity issue	Outcome
8	Serious failure	High severity problem	Outcome
12	Major failure	Critical error	Outcome

12. Backend Logging Architecture

In addition to the Supabase-based audit tables, the FastAPI backend implements structured JSON logging via **structlog**. A `RequestLoggingMiddleware` intercepts every HTTP request, assigns a unique `request_id`, and logs method, path, client IP, status code, and duration. Unhandled exceptions are captured at **ERROR** level with full traceback (`exc_info=True`) before being re-raised, so 500s are never invisible. The log level is configurable via the `LOG_LEVEL` environment variable (default: `INFO`). Log files use a **RotatingFileHandler** (10 MB per file, 5 backups — 50 MB max on disk), preventing unbounded disk growth. A **two-pass PII scrubbing pipeline** protects all log output — Pass 1 (field-name keywords, sync) runs in the structlog processor chain; Pass 2 (Microsoft Presidio NLP) runs **asynchronously** via `scrub_text()` and a `ThreadPoolExecutor(max_workers=2)` so the `asyncio` event loop is never blocked — see Section 13 for full detail. A startup warning is emitted if Presidio is unavailable. An `insert_audit_log()` helper in `backend/app/core/supabase.py` is called by all admin and pod activation routes to write rows into `audit_logs` with **ip_address**, **user_agent**, and a server-side **SHA-256 integrity_hash**. Audit writes are fire-and-forget — failures are silently swallowed so auditing never blocks the main request path.

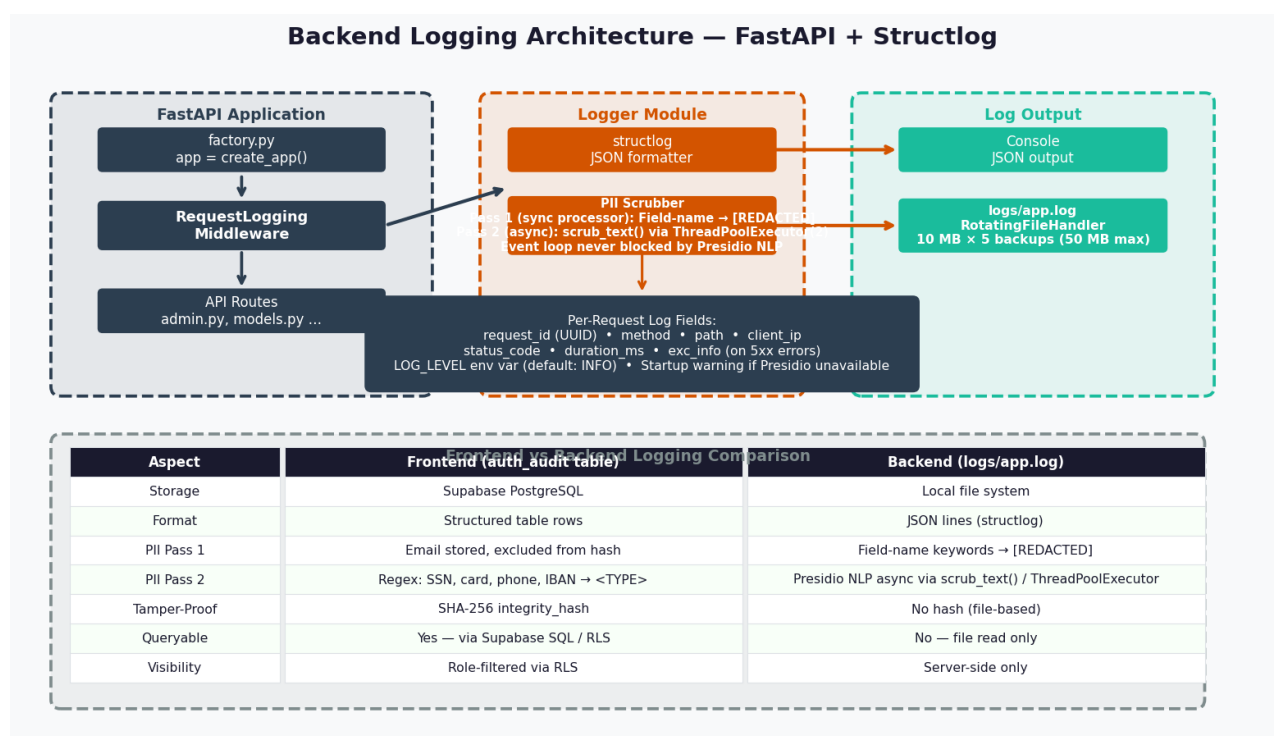


Figure 14 — Backend Logging Architecture

13. PII Detection — Presidio & Regex

All log data passes through a **two-pass PII scrubbing pipeline** before being written to any output. The pipeline runs in `backend/app/logger/__init__.py` (Python/Presidio) and is mirrored in `frontend/src/logger/index.ts` (TypeScript/regex).

Pass 1 — Field-name Keyword Match

Every key in the log payload is compared against a hard-coded set of 20 sensitive field names (case-insensitive). A match unconditionally replaces the value with [REDACTED] — regardless of the actual content. Fields covered: `password`, `passwd`, `secret`, `token`, `api_key`, `authorization`, `ssn`, `social_security`, `credit_card`, `card_number`, `cvv`, `dob`, `date_of_birth`, `birth_date`, `full_name`, `first_name`, `last_name`, `mobile`, `phone`, `phone_number`, `address`.

Pass 2 — Async Content-based PII Detection

After field-name scrubbing, string values can be scanned for PII content — catching sensitive data inside generic fields like `message` or `details` regardless of the key name. **Pass 2 does NOT run inside the sync structlog processor** — doing so would block the asyncio event loop under load. Instead, Pass 2 is exposed as `await scrub_text(value)`, an async helper that offloads Presidio NLP work to a dedicated `ThreadPoolExecutor(max_workers=2)` via `loop.run_in_executor()`. Call it from route handlers or middleware whenever you need to log potentially sensitive free-text content.

On the **backend**, Microsoft **Presidio** (`presidio-analyzer` + `presidio-anonymizer`) uses a **spaCy** English NLP model to detect named entities. Detected entities are replaced with labelled placeholders such as `<PERSON>`, `<US_SSN>`, `<CREDIT_CARD>`. Email addresses are pre-masked to `u***@domain` format before the Presidio scan. If Presidio or the spaCy model is unavailable, `_PRESIDIO_AVAILABLE` is set to `False`, a startup warning is logged, and the system degrades gracefully to Pass 1 only — no exception is raised.

On the **frontend**, `contentScrubString()` applies regex-based `PII_PATTERNS` that mirror Presidio entity labels, producing identical `<ENTITY_TYPE>` replacements. No ML model is required — the patterns run in-browser or in Node.js with zero additional dependencies.

Entity Type	Example Input	Replacement	Backend	Frontend
PERSON	John Smith	<PERSON>	Presidio NLP	—
EMAIL_ADDRESS	user@example.com	u***@example.com	Email mask + Presidio	Email mask + regex
PHONE_NUMBER	+1 (555) 123-4567	<PHONE_NUMBER>	Presidio NLP	Regex
CREDIT_CARD	4111 1111 1111 1111	<CREDIT_CARD>	Presidio NLP	Regex
US_SSN	123-45-6789	<US_SSN>	Presidio NLP	Regex
IBAN_CODE	GB29NWBK60161331926819	<IBAN_CODE>	Presidio NLP	Regex
LOCATION	New York, NY	<LOCATION>	Presidio NLP	—
IP_ADDRESS	192.168.1.1	(kept – forensic)	Intentionally excluded	—

Backend Installation

```
pip install presidio-analyzer presidio-anonymizer spacy && python -m spacy download en_core_web_sm
```

Or via `requirements.txt`: `presidio-analyzer>=2.2.0`, `presidio-anonymizer>=2.2.0`, `spacy>=3.4.0,<4.0.0`, `en-core-web-sm` @ <https://github.com/explosion/spacy-models/>...

PII Scrubbing Pipeline — Two-Pass Architecture

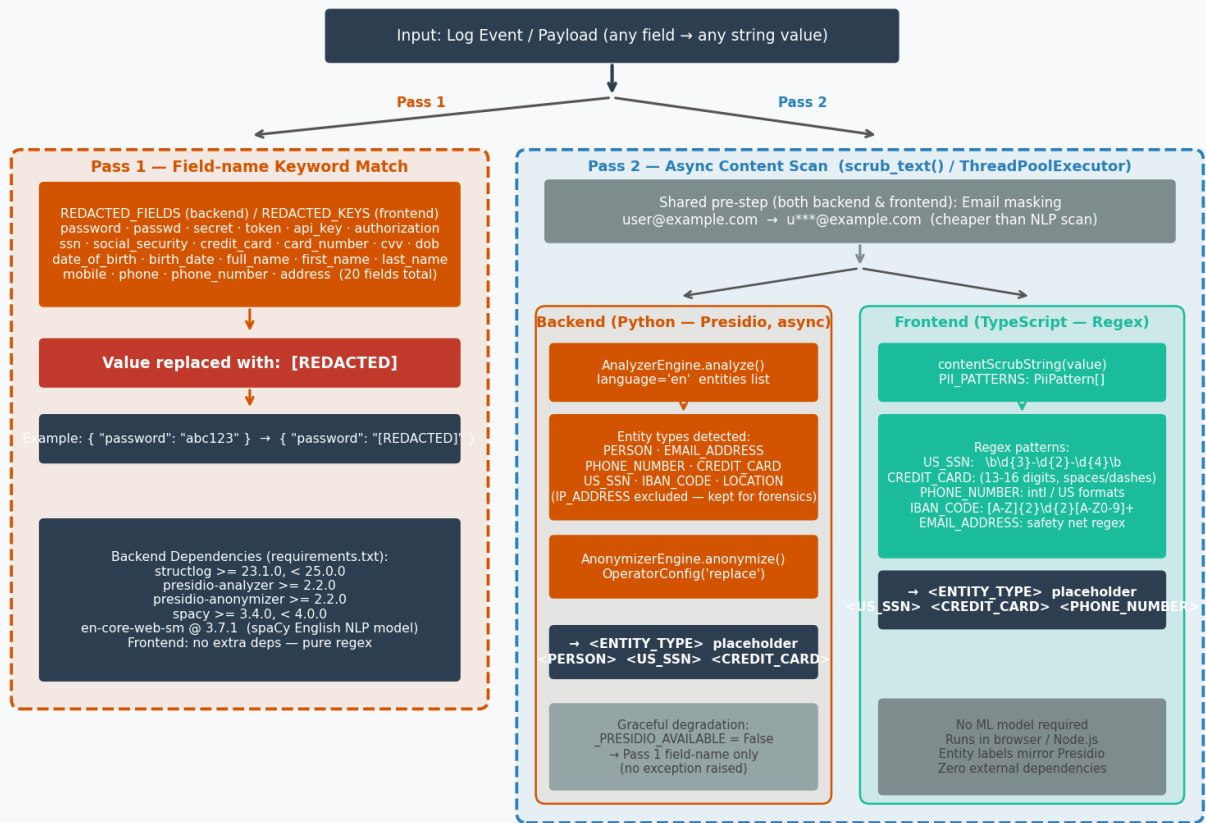


Figure 15 — PII Scrubbing Pipeline (Two-Pass Architecture)

14. File Reference

Key source files implementing the Activity Logs feature:

Category	File Path	Purpose
DB Migration	supabase/migrations/20260313091500_auth_audit_logs.sql	auth_audit table + RLS policies
DB Migration	supabase/migrations/20251116145514_03b8c065-31f2-46c1-a0b2-logs-table-368454.sql	audit_logs table + RLS policies
DB Migration	supabase/migrations/20260312193000_fnf9_schema_closure.sql	Role enum + user_roles + roles tables
DB Migration	supabase/migrations/20260312194000_fnf10_rls_hardening.sql	RLS hardening pass
DB Migration	supabase/migrations/20260316000000_auth_audit_user_id_fk_constraint.sql	FK constraint auth_audit.user_id -> auth.users(id)
DB Migration	supabase/migrations/20260316000001_audit_immutability.sql	Immutability triggers + integrity_hash on audit_logs + archive table
DB Migration	supabase/migrations/20260316000002_audit_logs_change_tracking.sql	previous_value + new_value JSONB columns + resource_id index
Frontend Page	frontend/src/app-pages/Activity.tsx	Activity log UI page
Frontend Lib	frontend/src/lib/auditHash.ts	SHA-256 integrity hash utility
Frontend Page	frontend/src/app-pages/Auth.tsx	Login/login_failed audit writer (resource_id: null)
Frontend Comp	frontend/src/components/Layout.tsx	Logout audit event writer (resource_id: null)
Frontend Comp	frontend/src/components/admin/PodActivationRequests.tsx	Pod approve/reject audit
Frontend Comp	frontend/src/components/AppSidebar.tsx	Role-filtered navigation
Frontend Hook	frontend/src/hooks/useUserRole.ts	Role detection hook
Backend	backend/app/core/supabase.py	insert_audit_log() helper (ip_address, user_agent, integrity_hash)
Backend	backend/app/routes/admin.py	Admin API endpoints + create_user/set_role audit calls
Backend	backend/app/routes/pod_activation.py	Pod activation endpoints + approve/reject/allocate audit calls
Backend	backend/app/logger/__init__.py	Structured request logging + two-pass Presidio PII scrubber
Backend	backend/requirements.txt	presidio-analyzer, presidio-anonymizer, spacy, en-core-web-sm
Frontend	frontend/src/logger/index.ts	Pino logger + two-pass PII scrubber (field-name + regex content)

15. Compliance & Immutability

The Activity Logs system is designed to meet the audit trail requirements of **EU AI Act (Art. 12/13)**, **SOC2 (CC7.2/CC9.1)**, and **NAIC AI Bulletin**. The following mechanisms are in place:

15.1 Database-Level Immutability

Migration 20260316000001_audit_immutability.sql installs four **BEFORE UPDATE / BEFORE DELETE** triggers on both `auth_audit` and `audit_logs`. The trigger function `prevent_audit_modification()` raises a P0001 exception for any modification attempt. This fires regardless of RLS bypass — `service_role`, backend calls, and direct Postgres connections are all blocked. Only a superuser running `ALTER TABLE ... DISABLE TRIGGER` can circumvent this, and that action is itself logged by Postgres.

Verified behavior in Supabase SQL Editor (role: postgres):

```
UPDATE public.auth_audit SET event = 'tampered' WHERE id = (...);
```

Result: **ERROR P0001: Audit log records are immutable (EU AI Act Art.12 / SOC2 CC7.2 / NAIC). Operation "UPDATE" on table "auth_audit" is not permitted.**

15.2 Integrity Hash Coverage

Both audit tables now carry `integrity_hash` (SHA-256). For **auth_audit**: computed in `auditHash.ts` (frontend, Web Crypto API) over 8 non-PII fields before insert. For **audit_logs**: computed in `insert_audit_log()` (backend, Python hashlib) over `user_id`, `action`, `resource_type`, `resource_id`, `created_at` before insert. Email and other PII are excluded from both hashes.

15.3 IP Address & User Agent Capture

All backend-initiated audit writes (admin actions, pod approvals/rejections, model allocations) populate `ip_address` from `request.client.host` and `user_agent` from the HTTP request header. This provides a complete forensic trail for regulatory review.

15.4 Retention Policy

EU AI Act requires logs retained for a minimum of **6 months**; SOC2 and NAIC require **12 months**. The following architecture is implemented via **pg_cron** (enabled in Supabase):

- **audit_logs_archive** and **auth_audit_archive** tables mirror the live tables with identical immutability triggers.
- A SECURITY DEFINER function (`archive_old_audit_logs()` / `archive_old_auth_audit()`) copies rows older than 12 months to the archive tables, then deletes from the live table. This is the only controlled exception to the immutability rule.
- A `pg_cron` job runs every Sunday at 02:00 UTC: `SELECT cron.schedule('archive-old-audit-logs', '0 2 * * 0', ...)`

15.5 Compliance Framework Coverage

Framework	Requirement	Implementation
EU AI Act Art.12/13	Immutable, traceable AI decision logs	Immutability triggers + <code>integrity_hash</code>
EU AI Act Art.12	Retain logs >= 6 months	<code>pg_cron</code> archival — rows never deleted

SOC2 CC7.2	No unauthorized modification	DB triggers block all UPDATE/DELETE
SOC2 CC9.1	Complete and accurate audit trail	ip_address + user_agent + integrity_hash
NAIC AI Bulletin	Full AI decision audit trail	approve_pod/reject_pod logged with details
NAIC AI Bulletin	Available for regulatory review	audit_logs_archive retained indefinitely

*This document was auto-generated from the Fideon OS codebase. All diagrams are drawn from live source code analysis.
Generated on 2026-03-16.*